

OJT PRODUCT REQUIREMENTS DOCUMENT

Student Name(s): Anurag Singh and Aditya Chauhan

Roll No(s): PST-25-0017 and PST-25-0195

Year & Section: 2026, Batch A and Batch B

Project Title (as assigned): AI Assisted Operating System (PsyDian)

Project Type: Operating System, Kernel, Gen AI

Stack / Framework: Rust, Cargo, Rustup, bootloader, core, alloc, x86_64, volatile, spin, lazy_static, uart_16550, pic8259, pc-keyboard, linked_list_allocator, QEMU, reqwest (or ureq), serde, serde_json

1. Problem Solving

1.1 What is the problem statement in your own words?

Modern operating systems have become increasingly powerful, supporting a wide range of hardware, applications, and services. However, despite decades of advancement, the primary method of interacting with these systems still requires users to understand technical concepts, command-line interfaces, system utilities, configuration files, and complex error messages. When users encounter issues such as application crashes, boot failures, driver conflicts, permission errors, memory faults, or unexpected system behavior, they are often expected to diagnose the problem by manually inspecting logs, searching through documentation, reading online forums, or copying commands from external sources without fully understanding their purpose. This approach is inefficient, time-consuming, and frequently results in incorrect configurations or additional system instability.

The challenge is not limited to beginners. Even experienced users and software developers spend a significant amount of time identifying the root cause of system failures, interpreting diagnostic information, and determining the appropriate recovery process. Existing operating systems generally provide logs and error messages but offer limited assistance in helping users understand what those messages mean or what actions should be taken next. As systems continue to increase in complexity, the gap between the information presented by the operating system and the user's ability to interpret that information continues to grow.

Recent advances in artificial intelligence have demonstrated the ability of large language models to understand natural language, analyze technical information, summarize complex data, and provide contextual guidance. However, in most existing operating systems, these capabilities are introduced as optional productivity features rather than being integrated into the core interaction model of the system.

As a result, users must still switch between the operating system, web browsers, documentation, search engines, and external AI tools to diagnose problems or complete routine system administration tasks.

There is an opportunity to redesign the interaction model of modern operating systems by incorporating artificial intelligence as a core system assistant while maintaining a secure separation between critical operating system components and AI services. Instead of requiring users to memorize commands, interpret cryptic diagnostic messages, or manually search for solutions, the operating system should be capable of understanding natural language requests, interpreting system state, analyzing structured diagnostic information, explaining failures in an understandable manner, recommending safe corrective actions, and assisting users with routine administrative tasks. Such an approach has the potential to reduce troubleshooting time, improve system accessibility, simplify day-to-day interaction, and create a more intuitive computing experience without compromising the stability, reliability, or security of the operating system itself.

1.2 Why does this problem exist or matter?

The complexity of modern operating systems has grown significantly over the past several decades. Originally designed for technical users and system administrators, operating systems gradually evolved to support millions of applications, diverse hardware architectures, networking capabilities, virtualization, security mechanisms, cloud services, and countless background processes. While these advancements have made operating systems more powerful, they have also increased the complexity of managing and troubleshooting them.

Most operating systems continue to expose low-level technical details directly to users. Error messages often describe what failed rather than why it failed or how it can be resolved. Diagnostic information is typically distributed across log files, terminal outputs, event viewers, and system utilities, requiring users to manually collect, interpret, and correlate information before identifying the root cause of a problem. This process assumes that users possess technical knowledge that many do not have.

Traditional operating systems were also designed around command-based interaction rather than intent-based interaction. Users are expected to know the exact syntax of commands, configuration options, file locations, and system utilities before performing administrative tasks. Even simple objectives often require multiple commands and familiarity with operating system internals. Consequently, users frequently rely on internet searches, documentation, community forums, or trial-and-error approaches instead of receiving contextual assistance directly from the operating system.

Another reason this problem persists is the separation between operating systems and modern artificial intelligence technologies. Existing operating systems generally treat AI as an external productivity feature rather than an integrated system capability. As a result, users must manually switch between the operating system and external AI tools or search engines to diagnose problems, understand system behavior, or obtain guidance. This fragmented workflow interrupts productivity and increases the time required to resolve issues.

Finally, operating systems prioritize stability, performance, and security over user guidance. Their primary responsibility is to manage hardware resources and execute software reliably, not to explain internal behavior or educate users about system operations. While this design philosophy has been successful for decades, advances in artificial intelligence create an opportunity to rethink how users interact with operating systems. By enabling systems to understand natural language, interpret diagnostic information, and provide contextual assistance while preserving security boundaries, future operating systems can become significantly more intuitive, accessible, and efficient without compromising their fundamental responsibilities.

1.3 Key inputs and expected outputs:

Note: this table illustrates the product's long-term interaction vision; Section 3 defines the actual 60-day MVP scope.

Input	Process	Output
Natural language request (e.g., "Open Downloads folder")	AI interprets user intent and converts it into valid operating system actions	Requested task is executed or the user receives confirmation before execution
Shell command entered by the user	Shell parses and validates the command before execution	Command executes successfully or returns an informative error message
Voice command	Speech-to-text converts speech into text, then forwards it to the command interpreter	Command is executed or AI requests clarification if the intent is ambiguous
Kernel panic or system crash log	Diagnostic component collects logs and sends them for AI analysis	Human-readable explanation of the issue with possible causes and recovery suggestions
System event logs	Log parser extracts relevant information and summarizes important events	Simplified diagnostic report with actionable recommendations
Unknown or incorrect command	Command validator detects invalid syntax and queries AI for suggestions	Correct command suggestions and explanation of the error
User asks a system-related question (e.g., "Why is Wi-Fi not working?")	AI analyzes available system information and context	Troubleshooting guidance with recommended safe actions
User requests automation (e.g., "Open VS Code every morning")	Automation engine validates and stores the requested workflow	Scheduled automation task is created successfully
System performance data (CPU, RAM, disk usage)	Monitoring service continuously analyzes resource usage	Performance summary, warnings, or optimization suggestions
User asks for command help (e.g., "How do I copy a file?")	AI retrieves command information and generates contextual guidance	Command syntax, explanation, examples, and safety notes
User configuration changes	Configuration manager validates and applies requested changes	Updated system configuration with confirmation or rollback on failure
Hardware events (keyboard, timer, device interrupts)	Kernel interrupt handler processes hardware events	Appropriate system response and updated device state

2. End Users / Potential Users

Category 1: Core Users (Daily Users)

- Everyday computer users who want a simpler and more intelligent way to interact with their operating system.
- Students who use computers for learning, programming, and academic work.
- Software developers who require AI-assisted debugging, command suggestions, and productivity features.
- Power users who frequently use terminal commands and system utilities.
- Users who prefer natural language or voice-based interaction over traditional command-line interfaces.
- Individuals with limited technical knowledge who need contextual guidance for troubleshooting and system management.

Category 2: Administrative User

- System administrators responsible for configuring, maintaining, and monitoring operating systems.
- IT support personnel who diagnose and resolve system issues for end users.
- Open-source contributors involved in developing, testing, and improving the operating system.
- Project maintainers responsible for releasing updates, managing documentation, and reviewing contributions.
- Researchers and engineers evaluating AI-assisted operating system architectures and intelligent system interaction.

3. Functional Scope

The following defines the actual 60-day MVP. Features not listed here are Stretch Goals (Section 8):

3.1 System Boot and Initialization

- Boot the operating system successfully on x86_64 architecture.
- Initialize the kernel and essential system components.
- Display system status during the boot process.

3.2 Kernel Management

- Manage CPU and memory resources.
- Handle hardware interrupts and exceptions.
- Provide panic handling and error reporting mechanisms.
- Maintain system stability and reliability.

3.3 Command-Line Shell

- Provide an interactive command-line interface for user interaction.
- Accept and execute built-in system commands.
- Maintain command history.
- Support command auto-completion and intelligent suggestions.

3.4 AI-Assisted Interaction

- Accept natural language requests from users.
- Interpret user intent and provide contextual responses.
- Suggest appropriate shell commands for user requests.
- Explain system errors and command failures in human-readable language.
- Analyze diagnostic logs and recommend safe recovery actions.

3.5 System Monitoring and Diagnostics

- Collect system logs and runtime information.
- Monitor important system events.
- Generate diagnostic reports for troubleshooting.
- Present meaningful error descriptions and recommendations

3.6 User Assistance

- Provide command usage guidance.
- Offer contextual help and documentation.
- Assist users in performing common operating system tasks.
- Recommend safe alternatives for invalid or potentially harmful commands.

3.7 Documentation and Developer Support

- Provide complete project documentation.
- Maintain developer documentation and architecture guides.
- Support project deployment, testing, and future extension.

4. System & Design Thinking

4.1 Sketch or describe your app flow / pipeline:

The system follows a modular architecture that separates critical operating system components from artificial intelligence services to maintain security, stability, and maintainability.

System Flow:

User provides voice input or text command → Shell receives and validates the request → Standard OS commands are executed by the kernel → AI-related requests are forwarded to the AI Bridge → AI Bridge collects system context and diagnostic logs → AI service analyzes the request and generates a response → AI Bridge validates the response → Shell displays AI suggestions or executes safe actions after user confirmation → Kernel continues managing hardware, memory, interrupts, and system resources independently.

4.2 What data structures or algorithms are central to this project?

The project relies on several fundamental data structures and algorithms to provide efficient system operation.

Data Structures

- Stack (function calls, interrupt handling, kernel execution)
- Heap (dynamic memory allocation)
- Linked List (heap allocator and memory management)
- Queues (keyboard input buffering and task scheduling)
- Ring Buffer (kernel logging and serial communication)
- Hash Map (command lookup and configuration management)
- Trees (page table hierarchy for virtual memory management)
- Strings and Buffers (command parsing and AI prompt generation)

Core Algorithms

- Memory allocation algorithms
- Paging and virtual memory translation
- Interrupt handling routines
- Command parsing and tokenization
- Auto-completion algorithm for shell commands
- AI prompt generation and response parsing
- Log analysis and diagnostic processing
- String matching and command suggestion algorithms
- Keyboard scan code decoding
- Simple scheduling and event processing mechanisms

4.3 How will you test correctness or performance?

The correctness, reliability, and performance of the system will be verified through continuous testing during development.

Functional Testing

- Verify successful boot on QEMU.
- Validate kernel initialization.
- Test memory allocation and deallocation.
- Verify interrupt handling.
- Test keyboard input processing.
- Validate shell command execution.
- Verify AI request and response workflow.

Integration Testing

- Verify communication between the shell and kernel.
- Verify communication between the shell and AI Bridge.
- Verify communication between the AI Bridge and external AI services.
- Validate complete request-response workflow.
-

Performance Testing

- Measure operating system boot time.
- Measure shell command execution latency.
- Measure AI response time.
- Monitor CPU utilization.

Stability Testing

- Trigger controlled kernel panics and verify panic handling.
- Test invalid and malformed user commands.
- Simulate communication failures with AI services.
- Validate recovery from runtime errors.

Success Criteria

The project will be considered successful if the operating system boots reliably, executes shell commands correctly, manages memory safely, handles hardware interrupts, communicates securely with the AI assistant, provides accurate AI-assisted diagnostics, and maintains stable operation throughout testing.

5. Timeline & Milestones

Week	Deliverables	Checkpoint
week 1	Project planning, PRD, SRS, repository setup, development environment configuration	Development environment is fully configured and project documentation is completed.
week 2	Bootloader integration, first bootable kernel, VGA text output	OS successfully boots in QEMU and displays output
week 3	GDT, IDT, exceptions, PIC setup, keyboard driver, timer interrupt	Kernel correctly handles interrupts and hardware input
week 4	Paging, frame allocator, heap allocator, alloc crate integration, kernel logger	Stable memory management with dynamic allocation and logging operational
week 5	Interactive shell, built-in commands, command parser, panic handler wired to logger	Kernel correctly handles interrupts and basic hardware interaction.
week 6	AiBridge architecture, structured logging, log parser, AI API abstraction, ai_help command, AI log explanation	AI receives system context and returns meaningful, accurate responses
week 7	Unit/integration/panic/stability testing, code refactoring, README, architecture docs	Core operating system functions are stable; documentation synchronized with implementation
week 8	PRD/SRS finalization, OJT report, presentation, demo script, GitHub cleanup, final bug fixing, viva prep and Documentation ready.	Project is production-ready for demonstration, evaluation, and submission

6. Risks & Dependencies

6.1 What is the hardest part technically right now?

The most challenging aspect of this project is operating system kernel development using Rust. Since this project involves low-level programming, it requires a deep understanding of computer architecture, memory management, interrupt handling, paging, boot processes, and hardware interaction. Debugging kernel-level code is significantly more difficult than debugging traditional applications because standard debugging tools and operating system services are unavailable during early kernel development.

Another major challenge is integrating artificial intelligence into the operating system while maintaining a secure separation between the kernel and AI services. Designing a reliable communication mechanism, processing system logs, interpreting AI responses, and ensuring that AI recommendations do not compromise system stability require careful architectural decisions. Time management is also a significant challenge, given the compressed 60-day timeline: the project combines learning advanced systems programming concepts with implementing a complete prototype in roughly two-thirds the time of a standard OJT cycle. Memory management (Week 4) is the single highest-risk stretch, any slippage there directly compresses the AI Bridge work in Week

6.2 What dependencies or help do you need from mentors?

The project will benefit from regular guidance from mentors in the following areas:

- Validation of the overall operating system architecture and design decisions.
- Guidance on Rust systems programming and kernel development best practices.
- Assistance in understanding memory management, paging, interrupts, and low-level debugging techniques.
- Feedback on integrating AI with operating system components while maintaining security and modularity.
- Periodic code reviews to improve code quality, maintainability, and performance.
- Suggestions for testing methodologies, debugging strategies, and project optimization.
- Feedback on technical documentation, project reports, and presentation materials to ensure the final deliverables meet academic and industry standards.

7. Evaluation Readiness

7.1 How will you prove that your project “works”?

The success of the project will be demonstrated through practical functionality, system testing, and end-to-end demonstrations rather than theoretical explanations. The operating system will be considered functional if it can successfully boot in the QEMU virtual environment, initialize the kernel, manage memory, handle hardware interrupts, and provide a stable command-line shell for user interaction.

To validate the AI-assisted features, demonstrations will show users interacting with the operating system using both traditional shell commands and natural language requests. The system will also be tested using simulated error scenarios, such as invalid commands and kernel diagnostic logs, to verify that the AI assistant can analyze system information, provide meaningful explanations, and recommend appropriate recovery actions. Additional testing will include memory allocation, interrupt handling, shell command execution, logging functionality, and complete integration between the operating system, AI communication layer, and external AI service. Successful completion of these demonstrations, along with stable execution during continuous testing, will confirm that the system operates as intended.

7.2 What success metric or goal will you aim for?

The project will be considered successful if it achieves the following measurable objectives:

- Successfully boots and runs on the QEMU emulator without critical failures.
- Initializes the kernel and core operating system components correctly.
- Executes built-in shell commands reliably.
- Handles keyboard input, interrupts, and memory management correctly.
- Maintains stable kernel logging and panic handling mechanisms.
- Successfully communicates with an external AI service through a secure userspace communication layer.
- Provides accurate AI-assisted diagnostic explanations via `ai_help`.
- Completes all planned MVP features within the 60-day project timeline.
- Produces complete technical documentation, project report, GitHub repository, and demonstration materials suitable for academic evaluation and future development.

8. Stretch Goals

The following features are considered future enhancements and are not part of the Minimum Viable Product (MVP). These features will be implemented only if sufficient time and resources are available after completing the core project objectives.

8.1 AI & Intelligence

- Offline AI support using local Large Language Models (LLMs).
- AI-powered predictive system diagnostics.
- Personalized AI assistant with long-term user preference memory.
- Intelligent workflow automation based on user behavior.
- Context-aware AI recommendations.
- AI-generated shell scripts and automation tasks.
- Multi-model AI provider support (OpenAI, Gemini, Claude, Local LLMs).

8.2 Advanced Voice Assistant

- Basic voice command support (accept spoken commands, convert to shell input).
- Real-time voice interaction with the operating system.
- Wake-word detection for hands-free activation.
- Continuous conversational mode without repeated activation.
- Context-aware voice conversations.
- Voice-controlled application and system management.
- Voice confirmation for privileged or destructive operations.
- Natural language task automation through voice commands.
- Speaker recognition and personalized voice profiles.
- Offline speech recognition and text-to-speech support.
- Dynamic conversational responses with contextual memory.

8.3 User Experience

- Graphical User Interface (GUI).
- AI-powered desktop assistant.
- Desktop notifications generated by AI.
- Intelligent search across files and applications.
- Customizable themes and appearance settings.

8.4 Operating System Features

- Basic file system implementation.
- Advanced process scheduling.
- Multi-user account management.
- User authentication and permission management.
- Basic networking support.
- USB device support.
- Power management and battery monitoring.
- System recovery and restore functionality.

8.5 Shell Enhancements

- Plugin architecture for custom commands.
- Advanced AI-powered auto-completion.
- Command aliases and macros.
- Interactive help system.
- Built-in package manager.
- Natural language shell execution.

9. Responsibilities

Anurag Singh	Aditya Chauhan	Mentor Notes
Project planning & requirement analysis	Project planning & requirement analysis	
System architecture & design	Architecture review & implementation support	
Bootloader development	AI Bridge development	
Kernel initialization	AI service integration	
Memory management & paging	AI prompt engineering	
Interrupt & exception handling	AI response processing	
Heap allocator & kernel logger	Natural language processing	
Command-line shell development	Logs Validating by AI	
Command parser & auto-completion	AI command suggestion module	
Kernel debugging & optimization	AI module testing & validation	
Documentation website	Testing & bug fixing	
GitHub repository & version control	Project report & presentation	
Final integration & deployment	Final testing & demonstration	

Signatures (Students):

Mentor:

Approval: Date: